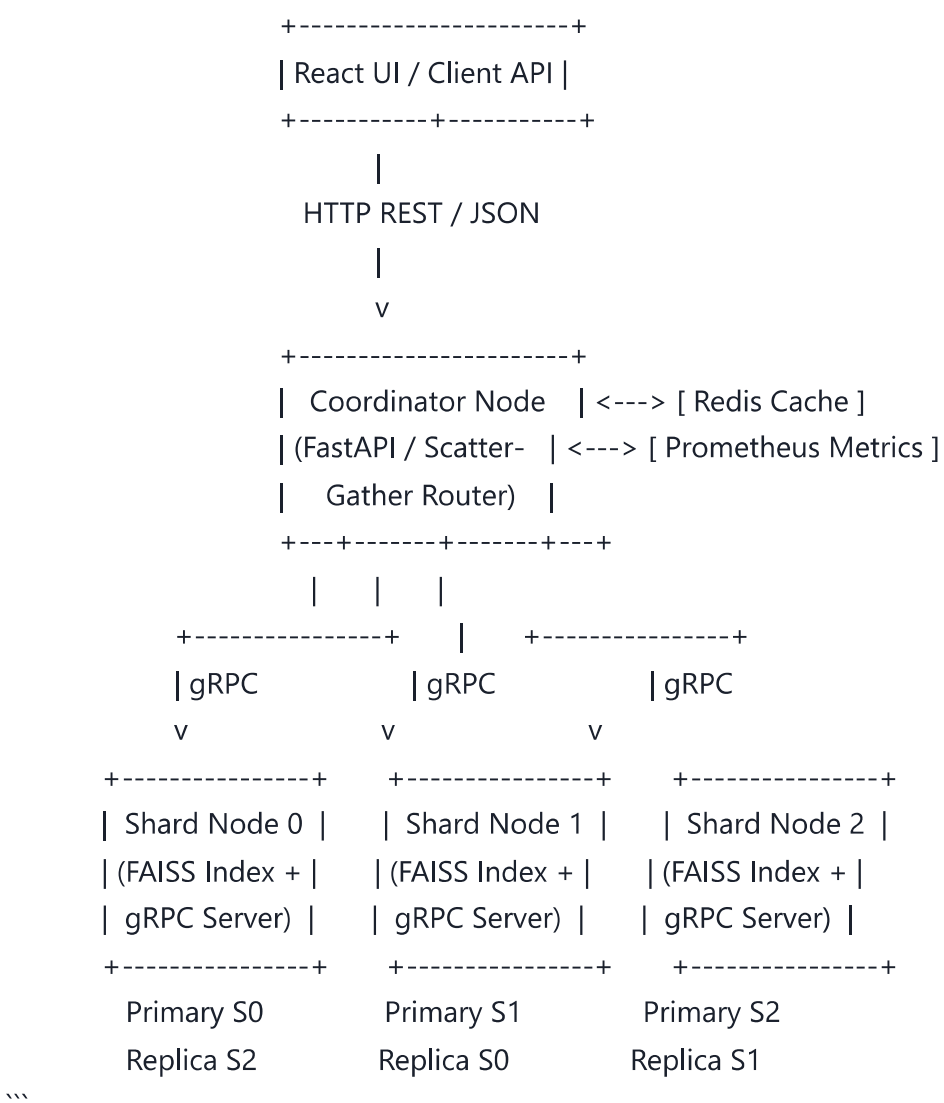
Distributed RAG System — Complete Architecture & File-by-File Technical Guide

This document provides a comprehensive technical breakdown of the **Distributed Retrieval-Augmented Generation (RAG)** system codebase, its individual microservice components, file responsibilities, data flow paths, and core distributed systems primitives.

---

1. System Overview & Data Flow Paths

The architecture separates concerns between **document ingestion (writing)** and **distributed scatter-gather querying (reading)** across a sharded vector database ring.



Ingestion Path (Writing Documents)
1. **Client Request**: The client posts a raw text document with a `doc_id` to `/api/v1/documents/ingest`.
2. **Chunking**: `DocumentProcessor` splits the document into overlapping chunks (500 chars with 50-char overlap) to preserve sentence boundaries.
3. **Local Embedding**: `EmbeddingsEngine` batch-embeds text chunks into 384-dimensional dense vectors using `all-MiniLM-L6-v2`. Vectors are L_2-normalized so dot products match exact cosine similarity.
4. **Hash Ring Placement**: `ConsistentHashRing` hashes `doc_id` to pick primary and replica target physical shards ($N=2$).
5. **Parallel gRPC Writing**: The coordinator sends parallel `IngestVector` gRPC calls to the target shard nodes.
6. **Local Indexing**: Each target shard node inserts the vector into its in-memory `FAISSVectorStore` (`IndexFlatIP`).
7. **Cache Invalidation**: The coordinator invalidates all Redis query cache entries so subsequent queries retrieve fresh context.

Query Path (Reading & LLM Synthesis)
1. **Client Request**: The client posts a prompt to `/api/v1/query`.
2. **Cache Check**: `QueryCache` checks Redis for an exact query hash match (`rag:cache:<md5>`). On a cache hit, the response returns in **< 2ms**.
3. **Query Embedding**: On a cache miss, the query text is converted into a 384-dim vector.
4. **Scatter-Gather Fan-out**: `ScatterGatherEngine` fans out `SearchVector` gRPC requests to all active shard nodes concurrently using `asyncio.gather`.
5. **Timeout Isolation**: Each gRPC call is wrapped in `asyncio.wait_for(timeout=1.5s)`. If a node is slow or dead, it is marked as `failed` without failing the overall request.
6. **Global Merge & Re-rank**: Candidate chunks from all surviving shards are gathered and sorted descending by similarity score. The top-K items are selected.
7. **LLM Synthesis**: `LLMGenerator` injects the top merged context snippets into a structured RAG prompt and queries the selected LLM provider (Google Gemini, Anthropic Claude, or local Mock fallback).
8. **Caching & Telemetry**: The coordinator caches the response in Redis, updates Prometheus metrics (`rag_query_latency_seconds`), and returns the response with per-shard execution telemetry.

---

2. Directory Structure & File Map

```text
Distributed RAG System/
```

```
├── shared/                    # Shared utility libraries across microservices
│   ├── config.py              # System configuration & environment variables
│   ├── document_processor.py  # Text chunker & DocumentChunk data model
│   ├── embeddings.py          # SentenceTransformers embedding engine
│   ├── faiss_store.py         # FAISS vector store wrapper with IndexIDMap2
│   ├── consistent_hash.py     # Hand-rolled Consistent Hash Ring with V-nodes
│   ├── llm.py                 # Pluggable LLM interface (Gemini/Claude/Mock)
│   └── telemetry.py           # OpenTelemetry tracing & Prometheus metrics
│
├── proto_gen/                 # Generated Protocol Buffer gRPC contracts
│   ├── vector_service.proto   # gRPC Protobuf API interface definitions
│   ├── vector_service_pb2.py  # Auto-generated Protobuf message classes
│   └── vector_service_pb2_grpc.py# Auto-generated gRPC client stubs & servicers
│
├── shard_node/                # Shard Vector Store Microservice
│   └── shard_service.py       # gRPC server managing local FAISS index per node
│
├── coordinator/               # Central Orchestrator & API Router
│   ├── coordinator_service.py # FastAPI Gateway, Health Check & Ring Manager
│   ├── scatter_gather.py      # Async parallel query fan-out router
│   └── cache.py               # Redis query result cache with TTL & invalidation
│
├── frontend/                  # Visual Web Console
│   └── index.html             # Single-page React UI with live cluster visualizer
│
├── tests/                     # Test Suites & Chaos Scripts
│   ├── unit/
│   │   ├── test_baseline.py   # Baseline single-node RAG unit tests
│   │   └── test_consistent_hash.py# Hash ring distribution & balance unit tests
│   ├── integration/
│   │   └── test_scatter_gather.py# gRPC fan-out & partial failure integration tests
│   ├── chaos/
│   │   └── chaos_test.py      # Mid-query node kill & self-healing chaos script
│   └── load/
│       └── locustfile.py      # Locust performance load test script
│
├── scripts/
│   └── seed_documents.py      # 1-click helper script to seed sample data
│
├── docker-compose.yml         # Multi-node cluster container orchestration
├── Dockerfile                 # Container image spec for Python services
```

```
├── requirements.txt             # System dependencies manifest
├── SYSTEM_ARCHITECTURE_GUIDE.md  # Comprehensive technical architecture guide
└── README.md                    # Technical README & setup manual
```

---

3. Deep Dive: Component & File Responsibilities

Module A: Shared Utilities (`shared/`)

1. [`shared/config.py`](file:///e:/job%20projects/Distributed%20RAG%20System/shared/config.py)
* **Purpose**: Single source of truth for global configuration.
* **Key Class**: `SystemConfig`
* **Responsibilities**:
  * Loads environment variables with sensible fallback defaults.
  * Defines embedding model (`all-MiniLM-L6-v2`), vector dimension (`384`), default chunk size (`500`), overlap (`50`), default `top_k` (`5`), and virtual nodes per shard (`100`).
  * Sets infrastructure connection strings (`REDIS_HOST`, `POSTGRES_URI`) and LLM API keys (`GEMINI_API_KEY`, `ANTHROPIC_API_KEY`).

2. [`shared/document_processor.py`](file:///e:/job%20projects/Distributed%20RAG%20System/shared/document_processor.py)
* **Purpose**: Text ingestion & chunking utility.
* **Key Classes**: `DocumentChunk`, `DocumentProcessor`
* **Responsibilities**:
  * `DocumentChunk`: Pydantic model holding `chunk_id`, `doc_id`, `text`, `chunk_index`, `metadata`, and `embedding`. Generates deterministic MD5 chunk hashes.
  * `DocumentProcessor`: Splits long texts into overlapping chunks while preserving word boundaries.

3. [`shared/embeddings.py`](file:///e:/job%20projects/Distributed%20RAG%20System/shared/embeddings.py)
* **Purpose**: Local vector embedding generator.
* **Key Class**: `EmbeddingsEngine`
* **Responsibilities**:
  * Lazy-loads `sentence_transformers.SentenceTransformer("all-MiniLM-L6-v2")`.
  * Provides `embed_text()` and `embed_batch()` methods returning normalized float32 NumPy arrays ($N \times 384$). L_2 normalization guarantees that inner products match exact cosine similarity.

4. [`shared/faiss_store.py`](file:///e:/job%20projects/Distributed%20RAG%20System/shared/faiss_store.py)

* **Purpose**: Vector database engine per shard.
* **Key Classes**: `SearchResult`, `FAISSVectorStore`
* **Responsibilities**:
  * Wraps FAISS `IndexFlatIP` inside `IndexIDMap2` to associate numeric uint64 internal IDs with vector embeddings.
  * Maintains thread-safe dictionaries (`id_to_chunk`, `chunk_id_to_int`) mapping internal integer IDs to full `DocumentChunk` metadata.
  * Implements `add_chunks()`, `search()` (top-K cosine search), and `delete_by_doc_id()`.

5. [`shared/consistent_hash.py`]
(file:///e:/job%20projects/Distributed%20RAG%20System/shared/consistent_hash.py)
* **Purpose**: Custom Consistent Hash Ring implementation.
* **Key Class**: `ConsistentHashRing`
* **Responsibilities**:
  * Maps physical node IDs to a 32-bit integer ring $[0, 2^{32}-1]$ via MD5 hashing.
  * Creates $V=100$ virtual nodes per physical shard (`shard_id#vnode_i`) to enforce balanced ring distribution ($\approx 33.33\% \pm 2\%$).
  * Provides `get_node(key)` and `get_nodes(key, replication_factor)` using binary search (`bisect_right`) to find clockwise target nodes.
  * Handles dynamic node addition and removal with minimal key remapping churn ($\approx 1/N$).

6. [`shared/llm.py`](file:///e:/job%20projects/Distributed%20RAG%20System/shared/llm.py)
* **Purpose**: Pluggable LLM generation interface.
* **Key Classes**: `RAGResponse`, `LLMGenerator`
* **Responsibilities**:
  * Formats retrieved top-K vector chunks into a structured context prompt.
  * Supports Google Gemini API (`gemini-2.5-flash`), Anthropic Claude API (`claude-3-5-sonnet`), and an offline `MockLLM` generator that synthesizes answers citing retrieved document sources.

7. [`shared/telemetry.py`]
(file:///e:/job%20projects/Distributed%20RAG%20System/shared/telemetry.py)
* **Purpose**: Prometheus metrics definitions & OpenTelemetry tracing setup.
* **Responsibilities**:
  * Defines Prometheus metrics: `rag_query_latency_seconds`, `rag_cache_hits_total`, `rag_shard_request_duration_seconds`, `rag_node_health_status`, `rag_documents_ingested_total`.
  * Configures OpenTelemetry `TracerProvider` and `BatchSpanProcessor`.

Module B: Protocol Buffers & gRPC Contract (`proto_gen/`)

1. [`proto_gen/vector_service.proto`]
(file:///e:/job%20projects/Distributed%20RAG%20System/proto_gen/vector_service.proto)
* **Purpose**: Protobuf schema for gRPC microservice RPCs.
* **RPC Methods Defined**:
  * `rpc IngestVector(IngestVectorRequest) returns (IngestVectorResponse)`
  * `rpc SearchVector(SearchRequest) returns (SearchResponse)`
  * `rpc HealthCheck(HealthCheckRequest) returns (HealthCheckResponse)`

2. `proto_gen/vector_service_pb2.py` & `proto_gen/vector_service_pb2_grpc.py`
* **Purpose**: Auto-generated Python gRPC stubs, serializers, and servicers compiled via `grpc_tools.protoc`.

Module C: Shard Microservice (`shard_node/`)

1. [`shard_node/shard_service.py`]
(file:///e:/job%20projects/Distributed%20RAG%20System/shard_node/shard_service.py)
* **Purpose**: gRPC server executing vector storage & retrieval on an isolated shard node.
* **Key Classes**: `VectorStoreServicer`
* **Responsibilities**:
  * Accepts command-line parameters (`--shard-id`, `--port`).
  * Runs a `grpc.aio.server` on the assigned port (e.g. 50051, 50052, 50053).
  * Executes vector insertion and similarity search on its local `FAISSVectorStore`.
  * Returns exact RPC execution timing (`latency_ms`) and health status.

Module D: Central Coordinator (`coordinator/`)

1. [`coordinator/coordinator_service.py`]
(file:///e:/job%20projects/Distributed%20RAG%20System/coordinator/coordinator_service.py)
* **Purpose**: API Gateway, Health Monitor, and Hash Ring Manager.
* **Key Class**: `CoordinatorState`
* **Responsibilities**:
  * Exposes FastAPI REST endpoints:
    * `POST /api/v1/documents/ingest`: Chunks, embeds, and routes write RPCs to primary + replica target shards via consistent hashing.
    * `POST /api/v1/query`: Checks Redis cache, triggers scatter-gather fan-out, calls LLM, and caches the result.
    * `GET /api/v1/cluster/status`: Returns node health, ring share %, and vector counts.

    * `GET /metrics`: Prometheus metrics scrape endpoint.
  * Runs a background heartbeat task every 2 seconds checking gRPC `HealthCheck`. Dynamically removes dead nodes from the hash ring and restores them upon recovery.

2. [`coordinator/scatter_gather.py`]
(file:///e:/job%20projects/Distributed%20RAG%20System/coordinator/scatter_gather.py)
* **Purpose**: Parallel scatter-gather query router with partial failure isolation.
* **Key Class**: `ScatterGatherEngine`
* **Responsibilities**:
  * Fans out `SearchVector` gRPC calls to all active shard nodes concurrently using `asyncio.gather`.
  * Enforces a 1.5-second per-shard timeout using `asyncio.wait_for()`.
  * Captures timed-out or dead nodes, flags `degraded: True`, and merges candidate chunks from surviving shards sorted by score.

3. [`coordinator/cache.py`]
(file:///e:/job%20projects/Distributed%20RAG%20System/coordinator/cache.py)
* **Purpose**: Redis query result cache.
* **Key Class**: `QueryCache`
* **Responsibilities**:
  * Generates MD5 query cache keys (`rag:cache:<md5>`).
  * Stores complete RAG responses with a 300-second TTL.
  * Implements explicit cache invalidation (`invalidate_all()`) triggered on document ingestion.
  * Falls back to an in-memory dictionary if Redis is offline.

Module E: Frontend Web Console (`frontend/`)

1. [`frontend/index.html`]
(file:///e:/job%20projects/Distributed%20RAG%20System/frontend/index.html)
* **Purpose**: Interactive single-page glassmorphism web UI built with React & Babel.
* **Responsibilities**:
  * **Interactive Chat Console**: Displays queries, synthesized answers, source chunk attributions, and cache/degraded status badges.
  * **Scatter-Gather Latency Inspector**: Renders total fan-out time and per-shard response metrics.
  * **Live Shard Topology Panel**: Displays real-time pulse health indicators (Green/Red), vector counts, and ring share % for `shard_0`, `shard_1`, and `shard_2`.
  * **Document Ingestion Interface**: Allows live text ingestion and hash ring distribution visualization.

Module F: Test Suites & Chaos Verification (`tests/`)

1. [`tests/unit/test_baseline.py`]
(file:///e:/job%20projects/Distributed%20RAG%20System/tests/unit/test_baseline.py)
* **Purpose**: Baseline single-node unit tests.
* **Verifies**: Document chunking boundaries, sentence-transformer embedding normalization, FAISS retrieval accuracy, and mock LLM synthesis.

2. [`tests/unit/test_consistent_hash.py`]
(file:///e:/job%20projects/Distributed%20RAG%20System/tests/unit/test_consistent_hash.py)
* **Purpose**: Hash ring distribution unit tests.
* **Verifies**: Virtual node balance ($\approx 33.33\% \pm 2\%$), primary/replica node separation, and minimal key remapping churn when nodes join/leave.

3. [`tests/integration/test_scatter_gather.py`]
(file:///e:/job%20projects/Distributed%20RAG%20System/tests/integration/test_scatter_gather.py)
* **Purpose**: Integration test for gRPC fan-out.
* **Verifies**: Ingestion & retrieval across real gRPC servers running on dynamic ports, and proper handling of unreachable/dead shard stubs.

4. [`tests/chaos/chaos_test.py`]
(file:///e:/job%20projects/Distributed%20RAG%20System/tests/chaos/chaos_test.py)
* **Purpose**: Mid-query node kill & recovery chaos simulation.
* **Verifies**:
  1. Starts 3 shard node subprocesses (`shard_0`, `shard_1`, `shard_2`).
  2. Ingests documents with $N=2$ replication.
  3. Sends a `SIGKILL` to `shard_1` mid-operation.
  4. Confirms heartbeat removes `shard_1` within 1s and queries succeed using replica nodes.
  5. Restores `shard_1` and confirms self-healing recovery.

5. [`tests/load/locustfile.py`]
(file:///e:/job%20projects/Distributed%20RAG%20System/tests/load/locustfile.py)
* **Purpose**: Locust load testing script.
* **Verifies**: Measures RPS, p_{95}, and p_{99} latencies under multi-user concurrency.

---

4. Core Distributed Systems Concepts & Interview Deep-Dive

1. Why Consistent Hashing over Static `hash % N`?
Under static `hash(key) % N`, changing $N \rightarrow N+1$ (adding a node) causes $\frac{N-1}{N}$

(75%–90%) of all keys to move, triggering cache invalidation storms and massive network re-indexing. Consistent hashing with virtual nodes ensures only $\frac{1}{N}$ ($\sim 25\%$) of keys migrate to the new node, keeping ring distribution balanced within $\pm 2\%$.

2. Partial Failure Handling in Scatter-Gather
In distributed retrieval, waiting for the slowest node degrades system p_{99} latency (the "straggler problem"). By wrapping each gRPC RPC in `asyncio.wait_for(timeout=1.5s)`, the coordinator isolates dead or lagging nodes, sets `degraded: True`, and returns the best merged results from all responsive shards.

3. CAP Theorem Tradeoffs ($N=2$)
* **AP Bias (Availability & Partition Tolerance)**: Search engines prioritize fast partial results over blocking queries for global linearizability.
* **Tunable Consistency**: Supports `FASTEST_REPLICA` ($R=1$) for lowest latency or `READ_QUORUM` ($R=2$) for strict read consistency.

4. Cache Invalidation Strategy
Redis caches exact query hashes with a 300s TTL. To prevent serving stale retrieval context when new documents are added, document ingestion triggers an explicit cache invalidation (`invalidate_all()`).

---

5. Summary Table of Key Microservice Ports

Component	Protocol	Port	Description
Coordinator API	HTTP REST	`8000`	Gateway, Hash Ring, Scatter-Gather Router
Shard Node 0	gRPC	`50051`	Vector Shard 0 (FAISS Index)
Shard Node 1	gRPC	`50052`	Vector Shard 1 (FAISS Index)
Shard Node 2	gRPC	`50053`	Vector Shard 2 (FAISS Index)
Redis	TCP	`6379`	Query Cache & Ingestion Queue
Prometheus	HTTP	`9090`	Metrics Scrape & Visualization Server